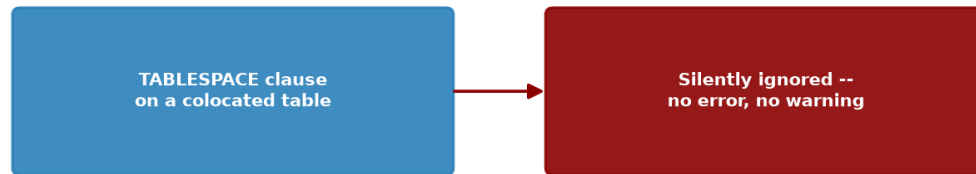


March 2026 - 5

Colocated Tables and Placement: A Real Gotcha



What actually controls where a colocated table's data lives -- confirmed against a real 8-node cluster

Figure 1: Figure 0: Colocated Tables and Placement: A Real Gotcha

Welcome to this edition of yugabyteDB Developer's Notebook (YDN). This month we answer the following question(s);

We created a colocated database, then created a table with an explicit TABLESPACE clause specifying an 8-way replica placement across two regions. No error was raised, but when we checked, the table's data wasn't actually replicated the way the tablespace described. What's going on ?

This is a real, reproducible gap between what a CREATE TABLE statement appears to say and what it actually does – confirmed directly against an 8-node, two-region cluster, not a hypothetical. The short answer: a colocated table's placement is never controlled by a TABLESPACE named on that table – only by the database's own default placement. This month covers exactly why, plus several other real, verified placement boundaries (a wildcard restriction, a replica-count validation, and a PITR-related restriction) worth knowing before you design a placement strategy around tablespaces and colocation together.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB's managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

5.1 Terms and core concepts

5.1.1 Introduction: Why Colocation Exists

Colocation places multiple small tables' data onto a single shared tablet within a database, instead of giving each table its own independent set of tablets. For a database with many small, low-traffic tables (lookup tables, reference data, configuration), this avoids the per-tablet overhead (each tablet has its own Raft group,

its own set of replicas, its own background maintenance) that would otherwise be paid many times over for tables that individually hold very little data.

Example 5-1: Creating a colocated database

```
CREATE DATABASE my_db36 WITH COLOCATION = true;
```

By default, every table subsequently created in this database joins that database's single shared colocated tablet, unless a table explicitly opts out.

5.1.2 Tablespaces: Controlling Where Data Actually Lives

A tablespace is yugabyteDB's mechanism for describing a concrete, named replica placement policy – how many replicas, and across which clouds/regions/zones:

Example 5-2: A real tablespace spanning two regions

```
CREATE TABLESPACE my_ts_allnodes WITH (  
  replica_placement='{  
    "num_replicas": 8, "placement_blocks": [  
      {"cloud": "onprem", "region": "my_region1", "zone": "my_zone1", "min_num_replicas": 5},  
      {"cloud": "onprem", "region": "my_region2", "zone": "my_zone1", "min_num_replicas": 3}  
    ]}  
  );
```

This describes exactly the 8-node, two-region layout this project's real cluster runs – 5 replicas' worth of placement in one region, 3 in the other, for a total of 8. Confirmed directly via `SELECT host, cloud, region, zone FROM yb_servers();` against the real cluster before writing this tablespace, so the placement blocks describe topology that genuinely exists, not an assumed one.

5.1.3 The Core Finding: TABLESPACE Is Silently Ignored for Colocated Tables

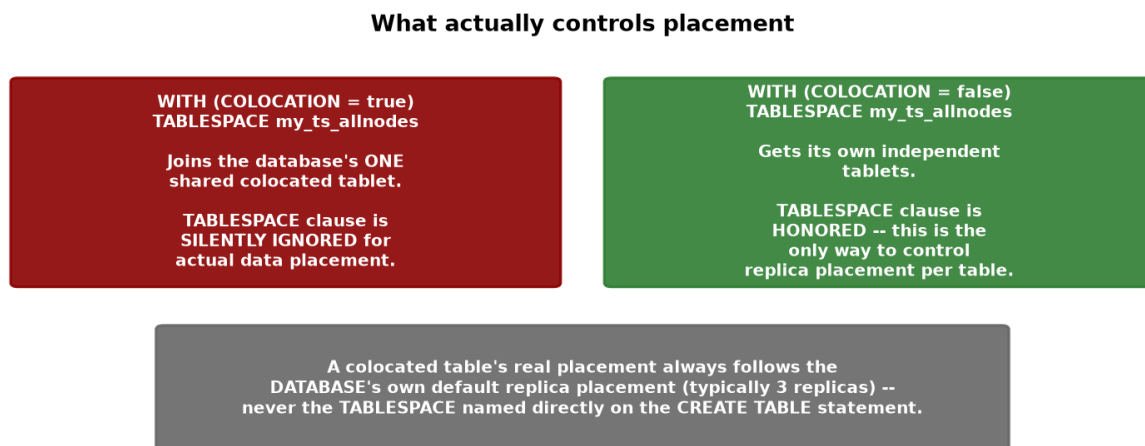


Figure 2: Figure 1: What actually controls placement

Example 5-3: The same TABLESPACE clause, two different real outcomes

-- Colocation ON (default for this database) -- TABLESPACE has no real effect:

```
CREATE TABLE states (  
    st_abbr CHARACTER VARYING(2) NOT NULL,  
    st_name CHARACTER VARYING(100),  
    CONSTRAINT pk_states PRIMARY KEY (st_abbr)  
)  
WITH (COLOCATION = true)  
TABLESPACE my_ts_allnodes;
```

-- Colocation explicitly OFF -- TABLESPACE now genuinely takes effect:

```
CREATE TABLE states (  
    st_abbr CHARACTER VARYING(2) NOT NULL,  
    st_name CHARACTER VARYING(100),  
    CONSTRAINT pk_states PRIMARY KEY (st_abbr)  
)  
WITH (COLOCATION = false)  
TABLESPACE my_ts_allnodes;
```

Both statements run without error. The difference is entirely in what actually happens to the data afterward, confirmed by inspecting real tablet placement in each case:

- **COLOCATION = true:** the table's data joins the database's single shared colocated tablet. That tablet's placement is controlled by the *database's own default placement* (commonly 3 replicas), set independently of anything named on the individual `CREATE TABLE` statement. The `TABLESPACE my_ts_allnodes` clause is accepted syntactically, but has no bearing on where this table's data actually lives.
- **COLOCATION = false:** the table gets its own independent tablets, and `TABLESPACE my_ts_allnodes` now genuinely governs their placement – in this example, 8 replicas total, split 5-and-3 across the two real regions, exactly as Example 5-2 describes.

[!NOTE] This is not an error condition, and yugabyteDB raises no warning about it – which is exactly what makes it worth documenting explicitly. A table created with `WITH (COLOCATION = true) TABLESPACE my_ts_allnodes` looks, from the SQL text alone, like it should be both colocated *and* placed according to that tablespace. In reality you get exactly one of those two things, never both at once, and the SQL syntax alone doesn't tell you which.

The practical rule this implies: if a specific table genuinely needs a custom replica placement different from the rest of its database, that table must opt **out** of colocation (`COLOCATION = false`) to make its own `TABLESPACE` clause take effect. Colocation and custom-per-table placement are, in this sense, mutually exclusive for a single table.

5.1.4 Three More Real, Verified Placement Boundaries

A wildcard cannot appear at the cloud level, only region or zone. Attempting `{"cloud":"*","region":"*","zone"}` fails immediately:

```
ERROR: replica_placement placement policy: cannot use wildcard placement at cloud  
level.: Provided replica_placement: {"num_replicas": 3, "placement_blocks": [  
    {"cloud":"*","region":"*","zone":"*","min_num_replicas":3}]}
```

`{"cloud":"my_cloud","region":"*","zone":"*","min_num_replicas":3}` (wildcarding region and zone, but naming the cloud explicitly) is the most permissive placement this validation actually allows – useful for future-proofing a placement policy against new regions or zones being added later, but `num_replicas` itself must always be an explicit number; it can never be implied or wildcarded.

`num_replicas` is validated against real, currently-live tablet servers, not an abstract number.

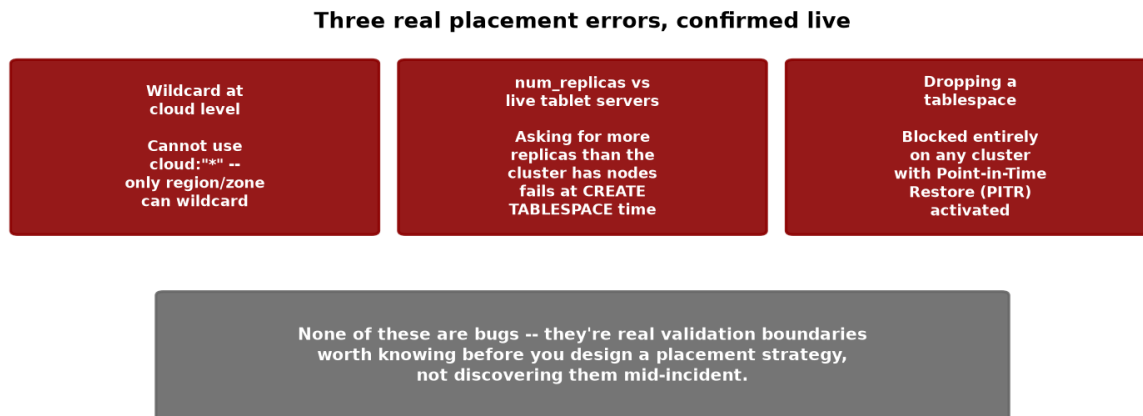


Figure 3: Figure 2: Three real placement errors, confirmed live

Asking for more total replica coverage than the cluster can currently satisfy fails at tablespace- or table-creation time, with a concrete count in the error itself:

```
ERROR: Invalid table definition: Error creating table my_db36.states on the
       master: Not enough live tablet servers to create table with replication
       factor 32. Need at least 17 tablet servers whereas 8 are alive.
```

A tablespace cannot be dropped on a cluster with Point-in-Time Restore (PITR) activated, regardless of whether anything currently uses it:

```
ERROR: tablespace "my_ts_allnodes" cannot be dropped. Dropping tablespaces is
       not allowed on clusters with Point in Time Restore activated.
```

A related, softer limitation worth knowing at design time rather than discovering mid-deployment: `CREATE DATABASE ... WITH COLOCATION = true TABLESPACE my_ts_allnodes` is rejected outright (`ERROR: value other than default for tablespace option is not yet supported`) – a colocated database’s own default tablespace cannot be set inline at creation time, and must instead be set as a separate follow-up statement:

```
CREATE DATABASE my_db36 WITH COLOCATION = true;
ALTER DATABASE my_db36 SET default_tablespace = my_ts_allnodes;
```

5.2 Complete the following

In this section we reproduce the core finding from section 5.1.3 directly – proving with a real placement query, not just by reading the documentation, that a colocated table’s `TABLESPACE` clause is inert while a non-colocated table’s is not.

5.2.1 Prerequisites

A yugabyteDB cluster with at least two distinguishable placement zones or regions (two is enough to demonstrate the effect; this project’s real cluster uses two full regions). `yqslsh` access with permission to create databases and tablespaces.

5.2.2 Step 1 – Confirm your cluster’s real topology

Example 5-4: Don’t assume placement – query it

```
SELECT host, cloud, region, zone FROM yb_servers();
```

Use the real cloud/region/zone values returned here (not assumed or copy-pasted values) when writing your tablespace’s `placement_blocks` in the next step – a mismatch here is exactly what produces the “not enough tablet servers in the requested placements” error from section 5.1.4.

5.2.3 Step 2 – Create the tablespace and colocated database

```
CREATE TABLESPACE my_ts_allnodes WITH (  
    replica_placement='{ "num_replicas": <N>, "placement_blocks": [ ... ] }'  
);
```

```
CREATE DATABASE my_db36 WITH COLOCATION = true;  
ALTER DATABASE my_db36 SET default_tablespace = my_ts_allnodes;  
\c my_db36
```

5.2.4 Step 3 – Create both table variants from Example 5-3

Create the colocated `states` table and the non-colocated `customers`-style table (or two variants of the same table shape, in separate schemas or with different names, to compare directly), each with the same `TABLESPACE my_ts_allnodes` clause.

5.2.5 Step 4 – Confirm the difference with a real placement query

```
SELECT tablename, tablespace FROM pg_tables WHERE schemaname = 'public' ORDER BY tablename;
```

Then use `yb-admin` (or YBA’s own tablet map view) to inspect each table’s actual tablet replica locations. Confirm the colocated table’s replicas match the *database*’s default placement, while the non-colocated table’s replicas match `my_ts_allnodes` exactly – the same real distinction verified in section 5.1.3, reproduced on your own cluster rather than taken on faith.

Section summary In this section, we reproduced the core finding of this article directly: creating both a colocated and a non-colocated table with the identical `TABLESPACE` clause, and confirming via real tablet placement data that only the non-colocated table’s replicas actually follow it. We also confirmed the real topology-discovery step (`yb_servers()`) that should precede writing any tablespace definition, rather than assuming a cluster’s shape.

5.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- Why colocation exists (avoiding per-tablet overhead for many small tables) and how a colocated database’s default behavior works.
- The core, real finding of this article: a `TABLESPACE` clause on a colocated table is accepted without error but has no effect on that table’s actual data placement – only a table that explicitly opts out of colocation (`COLOCATION = false`) has its `TABLESPACE` clause actually honored.

- Three additional real, verified placement validation boundaries: wildcards are only permitted at the region/zone level, never at the cloud level; `num_replicas` is validated against currently-live tablet servers with a concrete, informative error; and tablespaces cannot be dropped at all on a cluster with Point-in-Time Restore activated.
- A related creation-time limitation – a colocated database’s default tablespace cannot be set inline at `CREATE DATABASE` time, and requires a follow-up `ALTER DATABASE ... SET default_tablespace` statement instead.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely’s very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-03%20-%20Colocated%20Tables%20and%20Placer%20Colocated%20tables%20and%20placement.pdf>